

ListView com Dados Personalizados

Professor: Henrique Galati
IFSP - Campus Araraquara
Análise e Desenvolvimento de Sistemas

Na aula anterior, exploramos o componente **ListView** do Android, que permite apresentar uma lista rolável de elementos de forma simples e organizada. Estudamos como o **ListView** depende de três elementos principais: a **fonte de dados** (como um array ou **ArrayList**), o **adapter** (normalmente um **ArrayAdapter**), que atua como uma ponte entre os dados e a interface, e um **layout de item**, que define como cada elemento da lista será exibido. Utilizamos um exemplo com **ArrayAdapter** e o layout padrão **android.R.layout.simple_list_item_1**, mostrando como inicializar o adapter e associá-lo ao **ListView** via **View Binding**, além de configurar eventos de clique com o método **setOnClickListener**, exibindo o nome do item selecionado em um **Toast**. Encerramos com o exercício “**Catálogo de Cursos**”, que introduziu o uso de textos vindos do **strings.xml** e imagens personalizadas na lista.

Revisão rápida sobre Adapters

Em Android, um **Adapter** faz a ponte entre **uma fonte de dados** (ex: lista de objetos) e **um componente visual**, como **ListView**, **RecyclerView** (veremos futuramente) e **GridView**.

O Adapter **constrói dinamicamente** as views de cada item da lista, preenchendo com os dados corretos. O **ArrayAdapter** padrão serve para listas simples (ex: **List<String>**), onde cada item é só uma linha de texto.

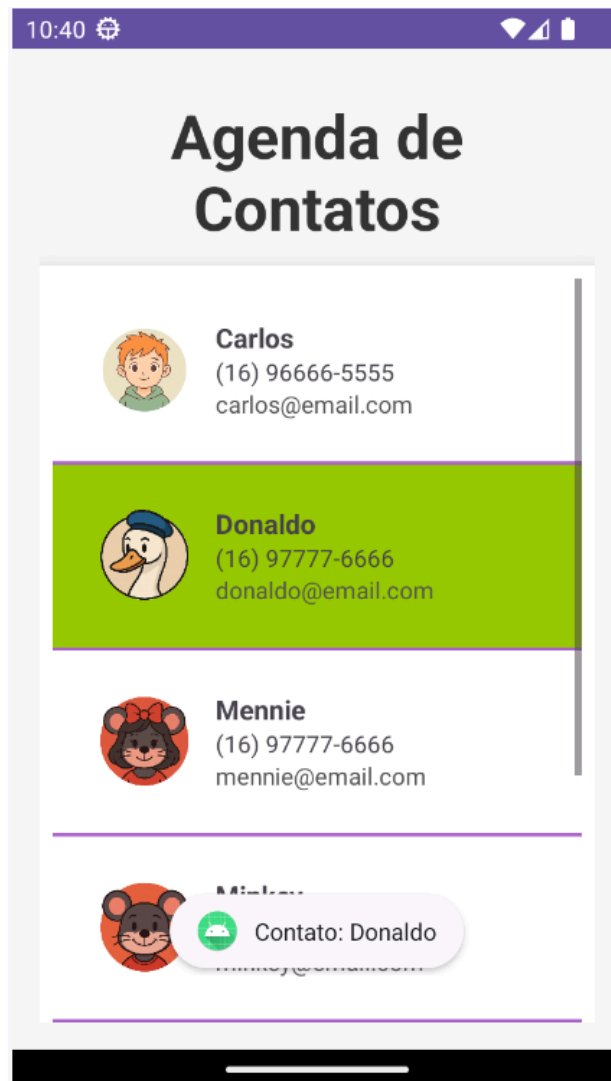
Mas quando você precisa mostrar **mais de uma informação** (ex: nome, telefone, e-mail), ou mostrar **imagens**, ou ainda usar um **layout customizado**, então é necessário **criar seu próprio Adapter**, estendendo a classe **ArrayAdapter<T>**.

Exemplo Prático: AgendaApp - Lista de contatos

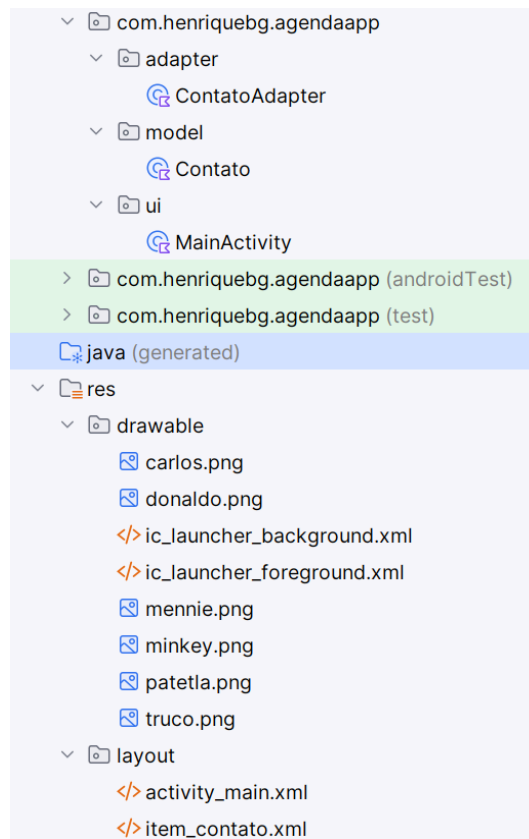
Vamos desenvolver um aplicativo chamado **AgendaApp**, que exibe uma **lista de contatos comerciais ou fictícios**. Cada item da lista deve mostrar:

- Foto do contato
- Nome completo
- Número de telefone
- Endereço de e-mail

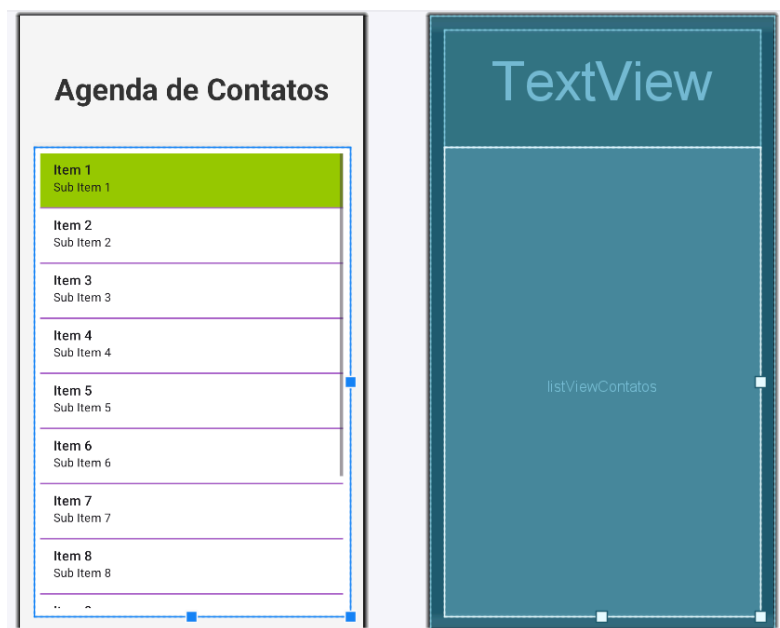
Ao clicar no nome do contato, o aplicativo deverá emitir um Toast com o nome do contato. Veja abaixo como a tela ficará. As imagens estão disponíveis no Moodle. Neste exemplo, ignore a configuração do `strings`, `color` e `themes`.



Estrutura do projeto



res/layout/activity_main.xml



```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#F9F9F9"
    android:padding="16dp"
    tools:context=".ui.MainActivity">

    <TextView
        android:id="@+id/tvTitulo"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="0.2"
        android:text="Agenda de Contatos"
        android:textSize="36sp"
        android:textStyle="bold"
        android:textColor="#333"
        android:gravity="center" />

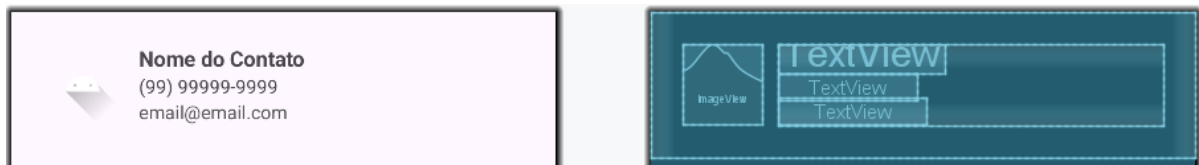
    <ListView
        android:id="@+id/listViewContatos"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="0.8"
        android:divider="@android:color/holo_purple"
        android:dividerHeight="2dp"
        android:background="@android:color/white"
        android:elevation="10dp"
        android:padding="8dp"
        android:listSelector="@android:color/holo_green_light" />
</LinearLayout>

```

Observe as linhas em destaque, são configurações novas para nossa **ListView**. Altere seus valores e perceba o papel de cada um na interface. Observe também que estamos usando **@android:color** ao invés de **@color**. Esta que contém android é uma paleta de cores que já vem pré-definidas pelo sistema.

res/layout/item_contato.xml

Este arquivo de layout será utilizado para renderizar cada contato (item) dentro da lista. No código, inflamos ele em cada item através do `layoutInflater`.



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="24dp">

    <ImageView
        android:id="@+id/imgFoto"
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:src="@drawable/ic_launcher_foreground"
        android:scaleType="centerCrop"
        android:layout_marginEnd="12dp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <TextView
            android:id="@+id/tvNome"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Nome do Contato"
            android:textSize="16sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/tvTelefone"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="(99) 99999-9999" />

        <TextView
            android:id="@+id/tvEmail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="email@email.com"
            android:textColor="#555" />

    </LinearLayout>
</LinearLayout>
```

model/Contato.kt

Vamos agora criar a classe modelo para usarmos em nossa `List<Contato>`. Perceba que a foto é armazenada como um inteiro, pois o Android salva **todos os recursos (imagens, strings, layouts, etc.)** no projeto como **inteiros constantes**, acessados através da classe R (para o caso de imagens, `R.drawable.nome_imagem`).

Relembrando: em Kotlin, usamos `data class` quando a principal função da classe é armazenar dados, como no caso de um contato com nome, telefone e e-mail. A `data class` já fornece automaticamente métodos úteis como `toString()`, `equals()`, `hashCode()` e `copy()`, o que evita código repetitivo e deixa o projeto mais limpo. Sempre que você estiver lidando com estruturas simples de dados, prefira `data class` em vez de `class` comum.

```
data class Contato(  
    val foto: Int,  
    val nome: String,  
    val telefone: String,  
    val email: String  
)
```

ui/MainActivity.kt

Agora, ao invés de usarmos uma `List<String>`, usaremos uma `List<Contato>`. Instanciamos a lista de contatos em `loadData`, ordenando de forma crescente através da lambda do nome do contato em `sortedBy{it.nome}`, com `it` representando cada elemento da lista. Criamos o Adapter personalizado para `Contato` e definimos como adapter da `ListView`.

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var binding: ActivityMainBinding  
    private lateinit var contatos: List<Contato>  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        loadData()  
        setupViews()  
        setupListeners()  
    }  
}
```

```

fun loadData(){
    contatos = listOf(
        Contato(
            R.drawable.minkey,
            "Minkey",
            "(16) 98888-7777",
            "minkey@email.com"
        ),
        Contato(
            R.drawable.mennie,
            "Mennie",
            "(16) 97777-6666",
            "mennie@email.com"
        ),
        Contato(
            R.drawable.patetla,
            "Patleta",
            "(16) 96666-5555",
            "patleta@email.com"
        ),
        Contato(
            R.drawable.truco,
            "Truco",
            "(16) 98888-7777",
            "truco@email.com"
        ),
        Contato(
            R.drawable.donaldo,
            "Donaldo",
            "(16) 97777-6666",
            "donaldo@email.com"
        ),
        Contato(
            R.drawable.carlos,
            "Carlos",
            "(16) 96666-5555",
            "carlos@email.com"
        )
    ).sortedBy { it.nome }
}

fun setupViews(){
    val adapter = ContatoAdapter(this, contatos)
    binding.listViewContatos.adapter = adapter
}

fun setupListeners(){
    binding.listViewContatos.setOnItemClickListener { _, _, position, _ ->
        val contato = contatos[position]
        Toast.makeText(this, "Contato: ${contato.nome}", Toast.LENGTH_SHORT).show()
    }
}
}

```

adapter/ContatoAdapter.kt

```
class ContatoAdapter(  
    private val context: Context,  
    private val lista: List<Contato>  
) : ArrayAdapter<Contato>(context, 0, lista) {  
  
    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {  
        val itemView = convertView ?: LayoutInflater.from(context)  
            .inflate(R.layout.item_contato, parent, false)  
  
        val contato = lista[position]  
  
        val imgFoto = itemView.findViewById<ImageView>(R.id.imgFoto)  
        val tvNome = itemView.findViewById<TextView>(R.id.tvNome)  
        val tvTelefone = itemView.findViewById<TextView>(R.id.tvTelefone)  
        val tvEmail = itemView.findViewById<TextView>(R.id.tvEmail)  
  
        imgFoto.setImageResource(contato.foto)  
        tvNome.text = contato.nome  
        tvTelefone.text = contato.telefone  
        tvEmail.text = contato.email  
  
        return itemView  
    }  
}
```

Estrutura básica do Adapter

Assinatura da Classe

```
class ContatoAdapter(  
    private val context: Context,  
    private val lista: List<Contato>  
) : ArrayAdapter<Contato>(context, 0, lista)
```

A classe estende `ArrayAdapter<Contato>`, indicando que ela trabalha com objetos do tipo `Contato`. No construtor da superclasse (`ArrayAdapter`), passamos: `context` (o contexto da tela), `0` (não estamos usando um layout padrão), `lista` (lista de contatos que será exibida).

Método getView()

```
override fun getView(position: Int, convertView: View?, parent: ViewGroup): View
```

O método `getView()` é a **função mais importante** em um `Adapter` no Android. É ele quem **cria e devolve a View visual de cada item da lista**, preenchendo os dados corretos para cada posição. Descrições dos parâmetros:

Parâmetro	Descrição
<code>position</code>	A posição do item na lista (ex: posição 0, 1, 2...). Use para acessar <code>lista[position]</code> .
<code>convertView</code>	Uma View antiga que saiu da tela e pode ser reaproveitada . Se for <code>null</code> , você precisa inflar uma nova.
<code>parent</code>	O componente pai (geralmente o <code>ListView</code>). Normalmente usado como parâmetro de layout ao inflar uma nova view.

O que deve ser feito dentro do `getView`:

1. Verificar se `convertView` é `null`:
 - Se sim, inflar o layout do item. **Inflar um layout** significa **transformar um arquivo XML de layout** (como `item_contato.xml`) em um **objeto de interface visível na tela (View)**.
 - Se não, reutilizar a view reciclada.

```
val view = convertView ?: LayoutInflater.from(context).inflate(R.layout.item_contato, parent, false)
```

2. Obter o objeto da lista correspondente à `position`.

```
val contato = lista[position]
```

3. Acessar os elementos da View (com `findViewById` ou `ViewBinding`).

```
val imgFoto = itemView.findViewById<ImageView>(R.id.imgFoto)
val tvNome = itemView.findViewById<TextView>(R.id.tvNome)
val tvTelefone = itemView.findViewById<TextView>(R.id.tvTelefone)
val tvEmail = itemView.findViewById<TextView>(R.id.tvEmail)
```

4. Preencher os dados do layout com os valores do objeto.

```
imgFoto.setImageResource(contato.foto)
```

```
tvNome.text = contato.nome
tvTelefone.text = contato.telefone
tvEmail.text = contato.email
```

5. Retornar a **View** pronta para ser exibida.

```
return itemView
```

Refatoração do ContatoAdapter para usar ViewBinding

```
class ContatoAdapter(
    context: Context,
    private val lista: List<Contato>
) : ArrayAdapter<Contato>(context, 0, lista) {

    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
        val binding: ItemContatoBinding
        val itemView: View

        if (convertView == null) {
            binding = ItemContatoBinding.inflate(LayoutInflater.from(context), parent, false)
            itemView = binding.root
            itemView.tag = binding
        } else {
            itemView = convertView
            binding = itemView.tag as ItemContatoBinding
        }

        val contato = lista[position]

        binding.imgFoto.setImageResource(contato.foto)
        binding.tvNome.text = contato.nome
        binding.tvTelefone.text = contato.telefone
        binding.tvEmail.text = contato.email

        return itemView
    }
}
```

Perceba que este novo código segue os mesmos passos do que deve ser feito em **getView**, porém utilizando o View Binding. A novidade aqui é o uso do atributo **tag** da classe **View**. Todo componente que herda de View (TextView, EditView, ImageView...) possui este atributo. Ele funciona como um “porta-treco”, ou seja, é uma **propriedade genérica** usada para **associar qualquer objeto** a essa view. Você pode pensar na **tag** como um **"campo extra" oculto** que acompanha a view, no qual você pode guardar qualquer dado útil para quando precisar dele mais tarde. Você pode guardar **strings**, **inteiros**, **objetos complexos** (como modelos de dados ou bindings), dentre outros.

No nosso caso, usamos a propriedade `tag` da `View` para **armazenar temporariamente o binding** associado ao layout do item. Ao guardar o `binding` em `view.tag` na primeira vez (quando `convertView == null` e é necessário inflar o layout) e recuperá-lo nas próximas com `view.tag as ItemContatoBinding`, garantimos melhor performance e evitamos chamadas desnecessárias ao `inflate()`. Isso permite **reaproveitar a mesma estrutura de views** quando o `ListView` recicla itens que saíram da tela, sem precisar inflar o layout ou recriar o binding novamente.